Distributed Scheduler - Part2 (@Scheduled Annotation)

Springboot does no magic, internally is just uses java: **ScheduledThreadPoolExecutor**

in other words:

Its just a wrapper above Java *ScheduledThreadPoolExecutor.*

1. **Enable Scheduling:**

@EnableScheduling:

- Enables Scheduling capability by loading all Scheduler related beans.
- Scan for @Scheduled annotations.

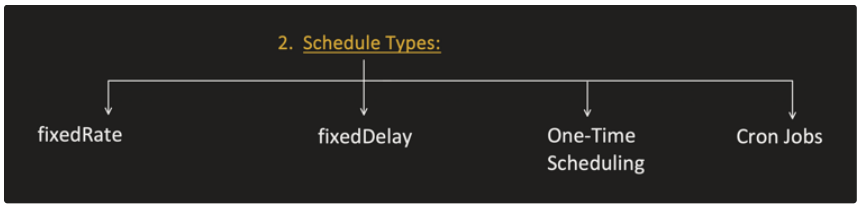
```
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class SchedulerApplication {

        public static void main(String[] args) {
            SpringApplication.run(SchedulerApplication.class, args);
        }
}

```

Concept && Coding



fixedRate:

- This is equivalent to method in java:

  [Already covered in-depth in PART-1]

  scheduleAtFixedRate(Runnable command, long initialDelay, long period, TimeUnit unit)

```
@Component
public class FixedRate {

    @Scheduled(initialDelay=3000, fixedRate = 5000)
    public void myJob() {
        System.out.println("Start: " + LocalTime.now());
       }
}

```

If we do not set initial delay, so by default:

initialDelay = 0 (can be started immediately)

But in above example, we have set it 3sec specifically.

Output:

```
2026-01-22T20:59:38.815+05:30  INFO 95121 --- [          main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8080 (
2026-01-22T20:59:38.819+05:30  INFO 95121 --- [          main] com.concepts.SchedulerApplication        : Started SchedulerApplication
Start: 20:59:41.833448
Start: 20:59:46.822779
Start: 20:59:51.823793
Start: 20:59:56.819134
Start: 21:00:01.824633
```

- First time Started after 3sec of application startup (its because of initial delay)
- After that, fixed 5 mins delay, calculated from the time, previous job started.

fixedDelay:

- This is equivalent to method in java:

  [Already covered in-depth in PART-1]
  scheduleWithFixedDelay(Runnable command, long initialDelay, long delay, TimeUnit unit)

```java
@Component
public class Scheduler{

    @Scheduled(initialDelay=1000, fixedDelay = 5000)
    public void myJob() {

         LocalTime startTime = LocalTime.now();
        try{
               Thread.sleep(3000);
        }
        catch (Exception e){
            e.printStackTrace();
        }
        LocalTime endTime = LocalTime.now();
        System.out.println("StartedAt:  " + startTime + " finishedAt: " + endTime);

       }
}

```

Concept && Coding

Output:

```
2026-01-22T21:42:31.213+05:30  INFO 95369 --- [          main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8080
2026-01-22T21:42:31.218+05:30  INFO 95369 --- [          main] com.concepts.SchedulerApplication        : Started SchedulerApplication
StartedAt: 21:42:32.228623 finishedAt: 21:42:35.233780
StartedAt: 21:42:40.241458 finishedAt: 21:42:43.246567
StartedAt: 21:42:48.251204 finishedAt: 21:42:51.254938
StartedAt: 21:42:56.260425 finishedAt: 21:42:59.262689
StartedAt: 21:43:04.266091 finishedAt: 21:43:07.267799
StartedAt: 21:43:12.273527 finishedAt: 21:43:15.276630
StartedAt: 21:43:20.278232 finishedAt: 21:43:23.283341
```

- First time Started after 1sec of application startup (its because of initial delay)
- After that, fixed 5 mins delay, calculated from the time, previous job finished.

One-Time Scheduling:

- This is equivalent to method in java:

[Already covered in-depth in PART-1]

schedule*(Runnable command, long delay, TimeUnit unit)*

```java
@Component
public class OneTimeSchedule{
  /*
  initialDelay field is mandatory, if we need to schedule the task only once.
  */
    @Scheduled(initialDelay=2000)
    public void myJob() {
        System.out.println("Start: " + LocalTime.now());
       }
}

```

Job, will run only once, after 2sec of initial delay.

<u>Cron Job:</u>

- We can say its, Repeated one-time scheduling.
- Based on wall clock time.

<u>Example of wall clock time:</u>

- Current time: **10:14**
- **every 3 minutes need to run the task**

Concept && Coding

Like fixedRate, it does not calculate next execution time by:

next execution time = previous execution start time + interval

first pick : 10:14

next execution time: 10:14 + 3 = 10:17

next execution time: 10:17 + 3 = 10:20

neither like fixedDelay, it does not calculate next execution time by:

next execution time = previous execution end time + interval

first pick : 10:14 → finished at 10:15

next execution time: 10:15 + 3 = 10:18 - which competes at 10:19

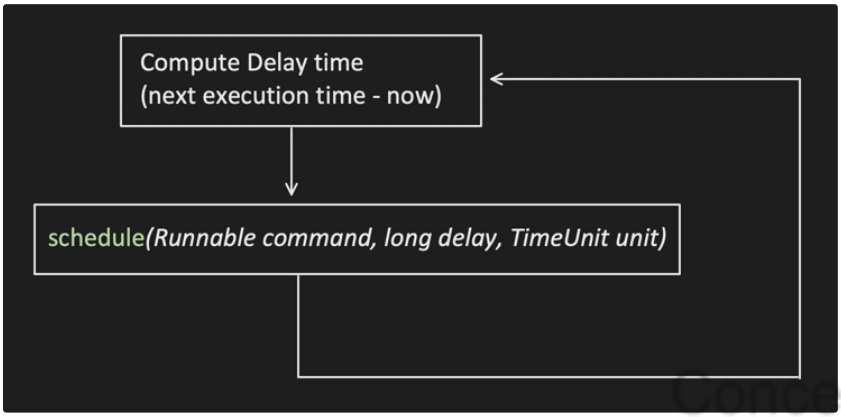
next execution time: 10:19 + 3 = 10:22

Cron works on wall clock time:

Valid cron minutes(for every 3 minutes run are):

00, 03, 06, 09, 12, 15, 18, 21, . . .

so after 10:14
if task gets completed on time, next run would be on 10:15, but if task takes longer say 10:16, then next run time is 10:18



Concept && Coding

Example:

Scenario:

- Cron: every **10 minutes**
- Task starts at **10:00**
- Task runs for **25 minutes**

expected cron times:

10:00

10:10

10:20

10:30

What Actually happens:

10:00 → task starts

10:10 → cron time occurs, but missed as task is still running

10:20 → cron time occurs, but missed as task is still running
10:25 → task finishes, it computes the next execution time.

startTime + every 10 minute
10:00 + 10 = 10:10 (missed)

10:10 + 10 = 10:20 (missed)

10:20 + 10 = 10:30 (future)

delay = 10:30 - 10:25 = 5 minutes goes to Timed_wait state.

10:30 → task starts

```java
@Component
public class CronSchedule {

    @Scheduled(cron = "0/3 * * * * *")
    public void myJob() {
        System.out.println("Start: " + LocalTime.now());
    }
}

```

Concept && Coding

Output:

```
2026-01-23T11:38:11.136+05:30  INFO 97316 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer
2026-01-23T11:38:11.141+05:30  INFO 97316 --- [           main] com.concepts.SchedulerApplication
Start: 11:38:12.014802
Start: 11:38:15.005413
Start: 11:38:18.001289
Start: 11:38:21.005382
Start: 11:38:24.004939
Start: 11:38:27.005523
Start: 11:38:30.002805
Start: 11:38:33.004603
Start: 11:38:36.005225
```

Cron Expression Syntax:

[second] [minute] [hour] [day of month] [month] [day of week]

second : 0-59

minute : 0-59

hour : 0-23

day of month : 1-31

month : 1-12 or JAN-DEC

day of week : 0-7 or SUN-SAT

0 represent SUN

7 also represent SUN

<u>Special characters:</u>

1. Asterisk (*) : means ***Every***

Like:
*if we put * in minute field, it runs every minute.*
*if we put * in hour field, it means every hour.*
likewise.

2.*List(,) : used to specify* ***multiple values****.*

Like:
0,20 in the seconds field means task runs at 0s and 20s only.

9,15 in the hours field means run at 9AM and 3PM only.

Concept && Coding

3. Range(-) : used to specify ***period***.

Like: 9-15 in the hours field means, "every hour from 9AM to 3PM)

4. Slash(/) : used to specify ***intervals***.

(start/step)

Like:

4/10 in second field means: start at 4sec, then keep adding 10 seconds (4,14,24,34,44,54)
*/10 in second field means 0, 10, 20, 30, 40, 50

5. **Last (L) : means Last**

Allowed only in:
Day-of-month : L in day of the month means Last day of month, Feb → 28 or 29, April → 30, December → 31

Day-of-week : used as DAYL means Last occurrence of given weekday in the month.

FRIL means last Friday of the month,

MONL means last Monday of the month.

6. **Don't know (?)** : when ? Is added, that specific field is ignored

Allowed only in:
Day-of-month
Day-of-week
Main use is to remove ambiguity.

Example:

[second] [minute] [hour] [day of month] [month] [day of week]
0 0 9 * * MON

Its ambiguous:

- Is it every day? or
- Every month Monday

0 0 9 ? * MON

- Now its clear, its runs at 9AM on Mondays
- Day field is ignored.

Concept && Coding

<u>Some Pratical example:</u>

1. **Every 5 seconds:**

[second] [minute] [hour] [day of month] [month] [day of week]

*/5 * * * * *

2. **Every 4 seconds:**

[second] [minute] [hour] [day of month] [month] [day of week]

- */4 * * * *

3. **Every day at 11:15 AM:**

[second] [minute] [hour] [day of month] [month] [day of week]

- 15 11 * * *

4. **Every weekday at 09:00 AM:**

[second] [minute] [hour] [day of month] [month] [day of week]

0 0 9 * * MON-FRI

5. **Every Sunday at midnight:**

[second] [minute] [hour] [day of month] [month] [day of week]

0 0 0 * * SUN

6. **Run at midnight on 1st day of every month:**

[second] [minute] [hour] [day of month] [month] [day of week]

0 0 0 1 * ?

7. **Last day of every month at midnight:**

[second] [minute] [hour] [day of month] [month] [day of week]

0 0 0 L * ?

Lets see common pitfalls while writing background job and its solutions:

Concept && Coding